

halfedge：布尔运算模块设计文档

src/boolean.rs

halfedge 库

2026-07-06

目录

1	模块定位	2
1.1	API 总览	2
2	BoolOp 分类规则	2
3	核心算法：Corefinement 四阶段	2
3.1	阶段 1：边-三角形求交	2
3.1.1	非共面情形：segment_triangle_intersection	3
3.1.2	共面情形：coplanar_segment_segment_3d	3
3.1.3	完整共面重叠：coplanar_triangle_overlap	3
3.2	阶段 2：三角形分裂	4
3.3	阶段 3：内外分类（多采样 + 内向偏移）	4
3.4	阶段 4：集合运算与重建	4
3.5	整体流程	5
3.6	数据丢弃警告	5
4	辅助函数	5
5	顶点去重	6
6	Panic 守卫	6
7	复杂度	6
8	限制	7
9	测试覆盖	8
10	设计取舍	8

1 模块定位

`boolean.rs` 在两个闭合流形三角网格 A 、 B 之上执行**集合运算**（并、交、差、对称差），返回新的 `MeshStorage`。

算法路线：Corefinement（共精化）。对每个三角形找边-三角形交点，在交点处精确切割三角形为子三角形，再对子三角形做内外分类后按布尔规则保留/丢弃。与旧射线法相比，`corefinement` 对跨越边界的三角形进行精确几何切割，使 $A - A$ **严格为空**（0 面），并提升部分重叠场景的几何精度。

1.1 API 总览

类别	API	语义
枚举	<code>BoolOp</code>	<code>Union/Intersection/Difference/SymmetricDifference</code>
核心	<code>boolean_operation(a, b, op)</code>	按 <code>op</code> 分派，返回新 <code>MeshStorage</code>
快捷	<code>boolean_union(a, b)</code>	$A \cup B$
	<code>boolean_intersection(a, b)</code>	$A \cap B$
	<code>boolean_difference(a, b)</code>	$A - B$
	<code>boolean_symmetric_difference(a, b)</code>	$A \triangle B$

2 BoolOp 分类规则

`classify(op, from_a, inside_other)` 根据子三角形来源与在另一网格内外的状态决定保留：

op	A 面 (from_a=true)	B 面 (from_a=false)
Union	<code>!inside_other</code>	<code>!inside_other</code>
Intersection	<code>inside_other</code>	<code>inside_other</code>
Difference	<code>!inside_other</code>	<code>false</code> （永不保留）
SymmetricDifference	<code>!inside_other</code>	<code>!inside_other</code>

语义：保留位于**结果边界上**的子三角形。例如并集保留 A 表面在 B 外部部分 + B 表面在 A 外部部分；差集仅保留 A 表面在 B 外部部分。这与 $A - A = \emptyset$ 的几何直觉一致： A 表面在 A 内部 $\Rightarrow$ `inside_other=true` $\Rightarrow$ `!inside_other=false` $\Rightarrow$ 全部丢弃。

3 核心算法：Corefinement 四阶段

3.1 阶段 1：边-三角形求交

对 A 的每个三角形 T ，遍历其三条边 e_k 与 B 的所有三角形 T' ，检测交点。分两种情形：

3.1.1 非共面情形: `segment_triangle_intersection`

设线段端点 p_0, p_1 , 三角形顶点 v_0, v_1, v_2 。用 Shewchuk 鲁棒 `orient3d` 谓词计算两端符号:

$$s_0 = \text{orient3d}(v_0, v_1, v_2, p_0), \quad s_1 = \text{orient3d}(v_0, v_1, v_2, p_1).$$

- 同号 $\Rightarrow$ 无交点;
- 共面 ($s_0 = s_1 = 0$) $\Rightarrow$ 转共面处理;
- 端点在平面上 ($s_0 = 0$ 或 $s_1 = 0$) $\Rightarrow$ 跳过, 避免重复顶点;
- 异号 $\Rightarrow$ 求交点参数:

$$t = \frac{s_0}{s_0 - s_1} \in (10^{-9}, 1 - 10^{-9}).$$

求得交点 $P = p_0 + t(p_1 - p_0)$ 后, 投影到三角形法向主轴的正交 2D 平面, 用 `point_in_triangle_2d` 判定是否在三角形内。

3.1.2 共面情形: `coplanar_segment_segment_3d`

当线段与三角形共面时, 对三角形的每条边做 2D 线段-线段求交。投影到两条边叉积的主轴正交平面, 用参数方程求解:

$$t_1 = \frac{(p_3 - p_1) \times d_2}{d_1 \times d_2}, \quad t_2 = \frac{(p_3 - p_1) \times d_1}{d_1 \times d_2},$$

其中 $d_1 = p_2 - p_1$, $d_2 = p_4 - p_3$ 。要求 $t_1, t_2 \in (10^{-9}, 1 - 10^{-9})$, 端点相交被过滤。

3.1.3 完整共面重叠: `coplanar_triangle_overlap`

当三角形 A 的三个顶点均在 B 的平面上时 (`orient3d` 均为 0), 两面完整共面。此时边-边求交无法捕捉面内包含情形 (如 B 整体落在 A 内部, 无边交叉), 需用 Sutherland-Hodgman 多边形裁剪计算重叠区域。

算法:

1. 投影到 2D (丢弃法向主轴, 与 `coplanar_segment_segment_3d` 一致);
2. 检测 B 的 2D 朝向, CW 时反转边方向确保 CCW;
3. 逐边将 A 裁剪到 B 内部 (Sutherland-Hodgman):
 - 对 B 的每条边 (e_s, e_e), 保留 A 中在边左侧 (CCW 内部) 的顶点, 计算进出交点;
 - 3 次裁剪后得到 $A \cap B$ 的凸多边形;
4. 利用平面方程 $\vec{n} \cdot \vec{x} + d = 0$ 将 2D 顶点反解回 3D。

约束点提取: 重叠多边形的顶点分为三类:

- A 的顶点在 B 内部 $\Rightarrow$ 已是 A 顶点, 无需额外处理;
- B 的顶点在 A 内部 $\Rightarrow$ 内部约束点, 加入 `interior_points` 列表;

- 边-边交点 $\Rightarrow$ 已在 A 的边上，加入对应边的交点列表。

内部约束点的分裂：在阶段 2 的耳裁剪三角化之后，对每个内部约束点 p ，定位包含 p 的子三角形 $T' = (v_0, v_1, v_2)$ ，做**扇形分裂**：

$$T' \rightarrow (v_0, v_1, p), (v_1, v_2, p), (v_2, v_0, p).$$

多个内部约束点逐个处理，每次在当前子三角形集合中定位并分裂。

3.2 阶段 2：三角形分裂

将每条边上的交点按参数 t 排序后插入三角形边界，形成多边形：

$$\text{polygon} = [v_0, \text{intersections on } e_0, v_1, \text{intersections on } e_1, v_2, \text{intersections on } e_2].$$

用 `ear_clipping_3d` 对多边形做耳裁剪三角化，得到子三角形列表。随后对每个**内部约束点**（来自共面重叠）做扇形分裂（见上节）。无交点且无内部约束点时返回原三角形。

3.3 阶段 3：内外分类（多采样 + 内向偏移）

对每个子三角形 T' 采样 7 个点：

$$\mathcal{S} = \{\text{重心}\} \cup \{3 \text{ 顶点}\} \cup \{3 \text{ 边中点}\}.$$

沿子三角形法向 $\hat{n}$ 的**内向偏移** ε ：

$$\vec{p}_{\text{sample}} = \vec{s} - \varepsilon \hat{n}, \quad \varepsilon = \frac{10^{-4}}{\|\vec{n}\|} \quad (\|\vec{n}\| < 10^{-14} \text{ 时回退 } 10^{-4}).$$

对每个采样点用射线奇偶法判定是否在另一网格内部，**多数表决** ($\geq 4/7$) $\Rightarrow$ `inside_other=true`。

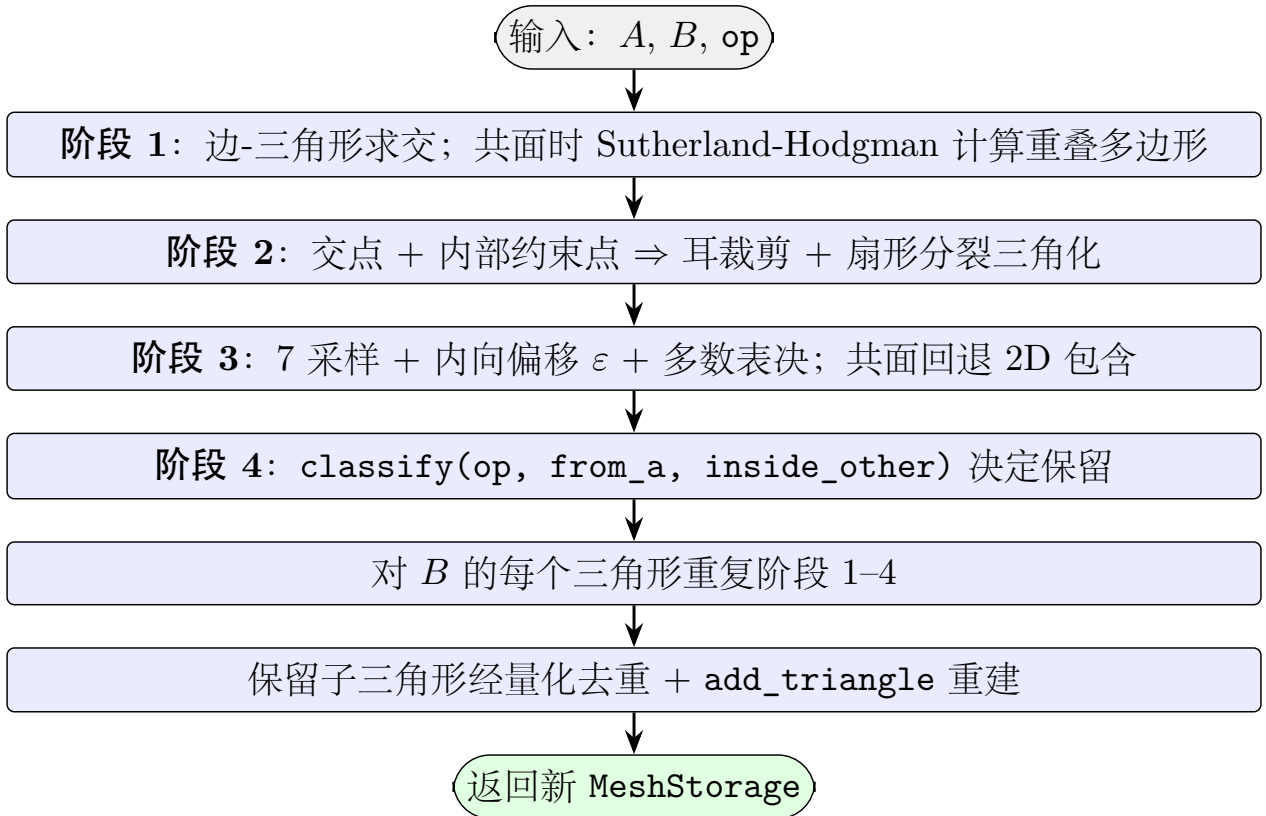
内向偏移的意义：对共面情形（如 $A - A$ ），三角形恰在另一网格表面，单侧采样会产生歧义。内向偏移确保采样点明确落入网格内部，使 $A - A$ 严格为空。

共面回退：当全部 7 个采样点均判定为外部 (`inside_count = 0`) 时，可能是**背对背共面**（ A 的面法向 $+z$ ， B 的面法向 $-z$ ），内向偏移使采样点离开对方体积。此时用 2D 包含检测回退：若子三角形重心与网格某面共面（`orient3d` 均为 0）且在该面的 2D 投影内，则判定为 **inside**。此回退确保并集/差集中背对背共面区域被正确丢弃，避免非流形结果。

3.4 阶段 4：集合运算与重建

对每个子三角形调用 `classify(op, from_a, inside_other)` 决定保留。保留的子三角形通过 `get_or_add_vertex`（量化 10^{-9} 去重）+ `add_triangle` 重建结果网格。

3.5 整体流程



3.6 数据丢弃警告

当 `add_triangle` 创建三角形失败时, 不再静默丢弃, 而是计数并在返回前通过 `eprintln!` 输出警告:

[halfedge::boolean] N

4 辅助函数

函数	签名	用途
<code>segment_triangle_intersection</code>	<code>(p0, p1, v0, v1, v2) -> Option<(V3, f64)></code>	非共面线段-三角形求交
<code>point_in_triangle_3d</code>	<code>(p, v0, v1, v2) -> bool</code>	投影 2D 后用 <code>point_in_triangle_2d</code> 判定
<code>coplanar_segment_segment_3d</code>	<code>(e1a, e1b, e2a, e2b) -> Option<(V3, f64, f64)></code>	共面线段-线段 2D 参数求交
<code>sutherland_hodgman_clip</code>	<code>(&[2D], 2D, 2D) -> Vec<[2D]></code>	Sutherland-Hodgman 多边形裁剪 (半平面)
<code>coplanar_triangle_overlap</code>	<code>(a: [V3;3], b: [V3;3]) -> Option<Vec<V3>></code>	共面三角形重叠多边形 (3D 顶点, CCW)
<code>is_point_strictly_inside_triangle</code>	<code>(p, tri) -> bool</code>	判定点在三角形内部 (排除边界)
<code>point_on_triangle_edge</code>	<code>(p, tri) -> Option<(usize, f64)></code>	找点所在边及参数 $t \in (0, 1)$
<code>find_containing_triangle</code>	<code>(tris, p) -> Option<usize></code>	在子三角形列表中定位包含 p 的三角形
<code>point_inside_mesh</code>	<code>(point, mesh) -> bool</code>	射线奇偶法 (+x 方向)
<code>is_triangle_inside_mesh</code>	<code>(tri, mesh, &Bvh) -> bool</code>	7 采样 + 内向偏移 + 多数表决; 共面回退 2D 包含
<code>collect_triangle_intersections</code>	<code>(tri, mesh, &Bvh) -> ([Vec<EdgeIntersection>; 3], Vec<V3>)</code>	收集三边交点 + 共面内部约束点; BVH 候选过滤
<code>split_triangle_by_intersections</code>	<code>(tri, edge_int, &[V3]) -> Vec<[V3; 3]></code>	插入交点 + 耳裁剪 + 内部约束点扇形分裂

5 顶点去重

`get_or_add_vertex` 通过量化网格去重:

$$\text{key} = (\lfloor 10^9 \cdot x \rfloor, \lfloor 10^9 \cdot y \rfloor, \lfloor 10^9 \cdot z \rfloor) \in \mathbb{Z}^3.$$

`HashMap<[i64;3], VertexId>` 缓存已添加顶点; 阈值 10^{-9} 内的位置视为同一点。这保证子三角形间的共享顶点 (含 `corefinement` 产生的交点) 能被识别, 使 `add_triangle` 自动配对 `twin`。

6 Panic 守卫

逐条检查可能导致 panic 的路径:

风险路径	守卫
空网格输入	<code>face_count() == 0</code> 时返回空结果
退化三角形 (共线/重合)	<code>is_triangle_degenerate_3d</code> 跳过
子三角形退化 (耳裁剪产物)	同上, 二次跳过
零长度法向	$\ \vec{n}\ < 10^{-14}$ 时 ε 回退 10^{-4}
端点重复交点	$t \in (10^{-9}, 1 - 10^{-9})$ 过滤
共面边平行	$ d_1 \times d_2 < 10^{-14}$ 返回 <code>None</code>
多边形顶点不足	<code>polygon.len() < 3</code> 返回原三角形
耳裁剪失败	<code>tri_indices.is_empty()</code> 返回原三角形
<code>partial_cmp NaN</code>	<code>unwrap_or(Equal)</code>

7 复杂度

设 $F_a = \text{mesh_a.face_count}()$, $F_b = \text{mesh_b.face_count}()$, K = 保留子三角形数。

BVH 加速 `boolean_operation` 在入口对每个网格构建一次 BVH (参见 `bvh.rs`), 阶段 1 与阶段 3 的内层扫描通过 `Bvh::faces_in_aabb` (候选过滤) 与 `Bvh::ray_count_hits` (射线奇偶计数) 加速。下表给出加速后的复杂度, k 为候选面数 (远小于 F)。

函数	时间复杂度
<code>segment_triangle_intersection</code>	$O(1)$
<code>coplanar_segment_segment_3d</code>	$O(1)$
<code>collect_triangle_intersections</code>	$O(\log F_b + k_b)$ (每边 BVH 候选 + 精筛)
<code>split_triangle_by_intersections</code>	$O(m)$ (m = 交点数, 耳裁剪)
<code>is_triangle_inside_mesh</code>	$O(7 \cdot (\log F_b + k_b))$ (7 采样 $\times$ BVH 射线)
<code>classify</code>	$O(1)$
<code>get_or_add_vertex</code>	$O(1)$ 均摊
<code>Bvh::build</code>	$O(F \log F)$ (入口一次性)
<code>boolean_operation</code>	$O((F_a + F_b) \log(F_a + F_b) + (F_a + F_b) \bar{k} \bar{m})$

对比: 未加速版本 `collect_triangle_intersections` 与 `is_triangle_inside_mesh` 均为 $O(F_b)$ 内层扫描, 整体 `boolean_operation` 退化为 $O(F_a \cdot F_b \cdot \bar{m})$ 。对千面量级网格 ($F_a = F_b = 1000$), 未加速 $\sim 10^9$ 次操作, BVH 加速后 $\sim 10^5$ 次操作, 约 $10^4 \times$ 加速。

8 限制

- 仅支持**闭合流形三角网格**; 非闭合网格的内外判定无定义;
- `add_triangle` 的 `Result` 被显式忽略 (`let _ = ...`), 拓扑错误被静默吞掉;
- 共面回退对交集运算可能产生非流形结果: 两个仅共面接触的网格做交集时, 两侧的共面面片均被保留 (各判定为 `inside`), 产生重复面。

9 测试覆盖

测试名	验证点
union_disjoint_cubes	不相交两立方体并集: $F = 24$ (无丢弃)
union_overlapping_cubes	重叠两立方体并集: $4 \leq F \leq 24$ (内部面被切割丢弃)
intersection_overlapping_cubes	重叠立方体交集非空 ($V > 0$)
intersection_disjoint_cubes	不相交立方体交集为空 ($F = 0$)
difference_self_is_empty	$A - A$ 严格空 ($F = 0$, corefinement 精确切割)
symmetric_difference_disjoint_equals_union	不相交时对称差面数 = 并集面数
intersection_contains_cube	两相同立方体交集 ≥ 6 面
coplanar_overlap_containment	大三角包含小三角 (共面): Some 重叠多边形面积正确
coplanar_overlap_partial	部分共面重叠 (共享一边): Some , 面积 > 0
coplanar_overlap_disjoint	共面但不相交: None
coplanar_overlap_non_coplanar	非共面: None
coplanar_difference_small_on_large	小立方体贴大立方体顶面做差集: 拓扑有效, 面数减少
coplanar_union_small_on_large	小立方体贴大立方体顶面做并集: 拓扑有效 (背对背共面被丢弃)
coplanar_intersection_small_on_large	小立方体贴大立方体顶面做交集: 结果非空 (仅共面接触)
空网格测试	
union_empty_meshes_returns_empty	双空网格并集: $F = 0$ (早退路径)
intersection_empty_meshes_returns_empty	双空网格交集: $F = 0$
difference_empty_meshes_returns_empty	双空网格差集: $F = 0$
union_empty_and_cube_returns_cube_face_count	空 $\cup$ cube: $F \geq 12$ (cube 表面在外部全保留)
difference_cube_and_empty_returns_cube	cube $-$ 空: $F \geq 12$ (Difference 仅保留 A 面在外部)
intersection_cube_and_empty_returns_empty	cube $\cap$ 空: $F = 0$ (空网格内外判定恒为 false)
完全重合测试	
union_identical_cubes	两相同 cube 并集: 拓扑有效 (内向偏移使表面互判 inside, 结果可能为空)
intersection_identical_cubes	两相同 cube 交集: $F \geq 6$ (已知非流形局限, 仅校验面数)
symmetric_difference_identical_cubes_is_empty	两相同 cube 对称差: $F = 0$
包含关系测试	
difference_large_minus_small	大 $[-1, 1]^3$ 减小 cube (边长 0.5): 拓扑有效
intersection_large_and_small_returns_smallish	大 $\cap$ 小: $F > 0$, 拓扑有效 (结果为小 cube 表面)
非轴对齐测试	
union_rotated_cubes	轴对齐 cube 与绕 Z 旋转 45° cube 并集: 拓扑有效

全模块共 **26** 个单元测试, 其中 7 个为共面三角形处理测试, 12 个为边界场景 (空网格、完全重合、包含关系、非轴对齐) 新增测试。

10 设计取舍

- **为何用 corefinement 而非纯射线法?** 旧射线法将三角形作为整体保留/丢弃, $A - A$ 残留最多 2 面。Corefinement 在边交点处精确切割, 使 $A - A$ 严格为空, 部分重叠场景几何更精确。
- **为何 7 采样 + 内向偏移?** 重心可能恰在另一网格表面 (共面情形), 单点采样产生歧义。7 采样覆盖面内多点, 内向偏移确保共面时采样点明确落入网格内部, 多数表决鲁棒。

- **为何共面使用 Sutherland-Hodgman 而非更通用的 2D 布尔？** Sutherland-Hodgman 仅支持凸裁剪多边形，对三角形 B 而言天然满足。相比 Greiner-Hormann 等通用算法，它实现简洁、数值稳定，且仅需 3 次裁剪即可得到 $A \cap B$ 的凸多边形。配合内部约束点的扇形分裂，能精确对齐两侧网格的共面重叠区域边界，使背对背共面（如小立方体贴大立方体顶面）的并集/差集产生流形结果。
- **为何需要共面回退？** 7 采样 + 内向偏移在背对背共面时失效： A 的面法向 $+z$ 、 B 的面法向 $-z$ ，内向偏移使采样点分别向 $\pm z$ 离开对方体积，全部判为外部 $\Rightarrow$ 两侧共面面片均被保留 $\Rightarrow$ 非流形。回退用 2D 包含检测将共面重叠区域正确判为 inside，避免此问题。
- **为何 add_triangle 的错误被忽略？** 布尔结果中可能出现退化三角形或重复面，add_triangle 会返回 Err。当前实现选择丢弃错误三角形继续，保证调用方拿到非空结果。
- **为何顶点量化去重？** 浮点位置直接比较不可靠（ 10^{-16} 误差）。量化到 10^{-9} 网格既保证共享顶点被识别，又允许微小数值漂移。